```
!pip install scikit-learn
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
Requirement already satisfied: scikit-learn in
/usr/local/lib/python3.11/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in
/usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in
/usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in
/usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.5.0)
```

```
credit_card_data = pd.read_csv(r'/creditcard1.csv')
```

```
---------------------------------------------------------------------
-----
FileNotFoundError                         Traceback (most recent call
last)
<ipython-input-4-e3cc7c51f6db> in <cell line: 0>()
----> 1 credit_card_data = pd.read_csv(r'/creditcard1.csv')

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py
in read_csv(filepath_or_buffer, sep, delimiter, header, names,
index_col, usecols, dtype, engine, converters, true_values,
false_values, skipinitialspace, skiprows, skipfooter, nrows,
na_values, keep_default_na, na_filter, verbose, skip_blank_lines,
parse_dates, infer_datetime_format, keep_date_col, date_parser,
date_format, dayfirst, cache_dates, iterator, chunksize, compression,
thousands, decimal, lineterminator, quotechar, quoting, doublequote,
escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines,
delim_whitespace, low_memory, memory_map, float_precision,
storage_options, dtype_backend)
   1024     kwds.update(kwds_defaults)
   1025
-> 1026     return _read(filepath_or_buffer, kwds)
   1027
   1028

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py
in _read(filepath_or_buffer, kwds)
    618
    619     # Create the parser.
--> 620     parser = TextFileReader(filepath_or_buffer, **kwds)
    621
```

```
    622     if chunksize or iterator:

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py
in __init__(self, f, engine, **kwds)
   1618
   1619         self.handles: IOHandles | None = None
-> 1620         self._engine = self._make_engine(f, self.engine)
   1621
   1622     def close(self) -> None:

/usr/local/lib/python3.11/dist-packages/pandas/io/parsers/readers.py
in _make_engine(self, f, engine)
   1878                 if "b" not in mode:
   1879                     mode += "b"
-> 1880             self.handles = get_handle(
   1881                 f,
   1882                 mode,

/usr/local/lib/python3.11/dist-packages/pandas/io/common.py in
get_handle(path_or_buf, mode, encoding, compression, memory_map,
is_text, errors, storage_options)
    871         if ioargs.encoding and "b" not in ioargs.mode:
    872             # Encoding
--> 873             handle = open(
    874                 handle,
    875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory:
'/creditcard1.csv'
```

```
credit_card_data.head(5)
```


```
{"type":"dataframe","variable_name":"credit_card_data"}
```


```
credit_card_data.tail(5)
```


```
{"type":"dataframe"}
```


```
#getting information about the dataset
credit_card_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
```

```
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB


# Display information about the dataset
credit_card_data.info()

# Drop all the rows with null values
credit_card_data = credit_card_data.dropna()

# Display information about the dataset after dropping null values
credit_card_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
```

```
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
```

```
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

```
credit_card_data.isnull().sum()
```

```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
```

```
Class     0
dtype: int64
```

```
# Checking the counts of legit and fraudulent transcations
credit_card_data['Class'].value_counts()
```

```
Class
0    284315
1       492
Name: count, dtype: int64
```

This dataset is highly imbalanced because in the above 2 targeted values,one of them is bigger than the other. so there will be difficulties for the model to recognise the fradulaent transactions. Legit transaction =0 Fraud transaction =1

```
# Separating the dataset for the analysis
Legit = credit_card_data[credit_card_data.Class == 0]
Fraud = credit_card_data[credit_card_data.Class == 1]
print(Legit.shape)
print(Fraud.shape)
```

```
(284315, 31)
(492, 31)
```

```
#Statistica measure of the data
Legit.Amount.describe()
```

```
count    284315.000000
mean         88.291022
std         250.105092
min           0.000000
25%           5.650000
50%          22.000000
75%          77.050000
max       25691.160000
Name: Amount, dtype: float64
```

```
Fraud.Amount.describe()
```

```
count     492.000000
mean      122.211321
std       256.683288
min         0.000000
25%         1.000000
50%         9.250000
75%       105.890000
max      2125.870000
Name: Amount, dtype: float64
```

```
# Now we will use the "Under smapling" method for building a sample
data set containing similar distribution of the legit and fraudulent
```

```
transcations.
# Number of fraudulent transaction=

legit_sample =Legit.sample(n=159)
legit_sample
```

{"type":"dataframe","variable_name":"legit_sample"}

```
new_dataset.head()
```

{"type":"dataframe","variable_name":"new_dataset"}

```
#concatinating 2 Dataframes
new_dataset = pd.concat((legit_sample, Fraud),axis=0)

# Now we will check the count of the legit and fraudulent transaction
from the class column
new_dataset['Class'].value_counts()
```

```
Class
1    492
0    159
Name: count, dtype: int64
```

```
# to check the sample data is good, we compare the mean value of 0 and
1 which is legit and Fraudulent transaction..The value is very close
to each other , the sample data is good.
new_dataset.groupby('Class').mean()
```

{"type":"dataframe"}

```
# Spliting the data into features and targets
X = new_dataset.drop(columns='Class', axis=1)
Y = new_dataset['Class']

print(X)
```

```
            Time        V1        V2        V3        V4        V5
V6  \
153740   99945.0 -0.437301  0.863142  0.165989  0.069390  1.049284 -
1.128744
30681    36035.0 -1.270388  2.640482  0.433502  3.880280  0.510688
0.395098
101278   67780.0 -1.169074  0.271745  1.240351  1.129598  0.189616 -
0.970103
162090  114812.0 -2.241524 -1.150949 -0.386193  1.785847  2.600470 -
1.652676
165240  117301.0  2.136329  0.000237 -1.477615  0.161801  0.458914 -
0.593401
...          ...       ...       ...       ...       ...       ...
```

```
...
279863  169142.0 -1.927883  1.125653 -4.518331  1.749293 -1.566487 -
2.010494
280143  169347.0  1.378559  1.289381 -5.004247  1.411850  0.442581 -
1.326536
280149  169351.0 -0.676143  1.126366 -2.213700  0.468308 -1.120541 -
0.003346
281144  169966.0 -3.113832  0.585864 -5.399730  1.817092 -0.840618 -
2.943548
281674  170348.0  1.991976  0.158476 -2.583441  0.408670  1.151147 -
0.096695

               V7        V8        V9  ...       V20       V21
V22  \
153740  0.647095 -0.283385  1.359534  ... -0.195004 -0.125000
0.098623
30681   0.968948 -0.166148 -0.637016  ...  1.185791 -0.092335
0.659878
101278 -0.260870  0.526554 -0.956673  ...  0.163454  0.034353 -
0.415079
162090 -0.401608 -0.383147  0.638958  ... -0.903110 -0.197579
1.011641
165240  0.235639 -0.329684  0.449529  ... -0.121682 -0.345223 -
0.802212
...          ...       ...       ...  ...       ...       ...        ..
.
279863 -0.882850  0.697211 -2.064945  ...  1.252967  0.778584 -
0.319189
280143 -1.413170  0.248525 -1.127396  ...  0.226138  0.370612
0.028234
280149 -2.234739  1.210158 -0.652250  ...  0.247968  0.751826
0.834108
281144 -2.208002  1.058733 -1.632333  ...  0.306271  0.583276 -
0.269209
281674  0.223050 -0.068384  0.577829  ... -0.017652 -0.164350 -
0.295135

              V23       V24       V25       V26       V27       V28
Amount
153740 -0.363930 -0.132629  0.164102  0.594607 -0.125921  0.029349
6.99
30681   0.139570  0.163485 -0.992879  0.174042  0.917713  0.297955
6.27
101278  0.051452  0.505046 -0.186414 -0.607798 -0.055965 -0.183919
9.99
162090 -0.930949  0.063340 -1.085910 -0.515015  0.850633  0.163786
100.86
165240  0.198795 -0.974153 -0.123178  0.256783 -0.065045 -0.068165
1.78
```

```
...          ...       ...       ...       ...       ...       ...
...
279863  0.639419 -0.294885  0.537503  0.788395  0.292680  0.147968
390.00
280143 -0.145640 -0.081049  0.521875  0.739467  0.389152  0.186637
0.76
280149  0.190944  0.032070 -0.739695  0.471111  0.385107  0.194361
77.89
281144 -0.456108 -0.183659 -0.328168  0.606116  0.884876 -0.253700
245.00
281674 -0.072173 -0.450261  0.313267 -0.289617  0.002988 -0.015309
42.53

[651 rows x 30 columns]
```

```
print(Y)
```

```
153740    0
30681     0
101278    0
162090    0
165240    0
         ..
279863    1
280143    1
280149    1
281144    1
281674    1
Name: Class, Length: 651, dtype: int64
```

```
#Spliting the data into training data and testing data
X_train, X_test, y_train, Y_test = train_test_split(X, Y,
test_size=0.2, stratify=Y, random_state=2)
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(651, 30) (520, 30) (131, 30)
```

MODEL Training Logistic Regression

```
model = LogisticRegression()

# training the Logistic Regression model with training data
model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/
_logistic.py:458: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as
```

```
shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(

LogisticRegression()
```

MODEL EVALUATION ACCURACY SCORE

```
# accuracy on training data
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, y_train)

print('Accuracy on training data : ', training_data_accuracy)

Accuracy on training data :  0.9365384615384615

#accuracy on the test data
X_test_prediction = model.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)

print('Accuracy score on test data : ', test_data_accuracy)

Accuracy score on test data :  0.9236641221374046

from google.colab import files
files.download('ML_PROJECT.csv')  # replace with your file name
```